

Даалгавар 2: Data Scrapping

Task:

Өөрийн сонгон авсан системээс өгөгдлүүдийг гарган авах data scrapping code бичнэ. Data scrap хийхдээ BeautifulSoup and Google colab ашиглана.

Бүх датаг цуглуулаад өгөгдөлөө tsv өргөтгөлтэй хадгалаад явуулна. Гарган авч буй өгөгдөлд тавигдах гол шаардлага нь дор хаяж 7-н баганатай байхыг шаардана. Үүнд линк болон unique identifier орохгүй. Хэрвээ өгөгдөл нь том хэмжээтэй бол Google drive руу хуулаад линкээ явуулна.

1. Сонгон авсан систем: Arxiv

[Arxiv](#) системээс гараас оноосон түлхүүр үгийн дагуу буцаасан эрдэм шинжилгээний өгүүллэгүүдийн мэдээллийг хуулах. Хуулсан мэдээлэлд дараах орно:

1. Arxiv code
2. Гарчиг
3. Зохиолчид
4. Тойм
5. Хэвлэгдсэн сэтгүүл
6. Илгээсэн огноо
7. Зарлагдсан огноо
8. Сэдвүүд

2. Google Scholar

[Google Scholar](#) системээс 1-р алхамд хуулсан бүх зохиолчдын мэдээллийг хуулах. Эдгээр нь:

1. Судалгааны чиглэл
2. Сургууль

3. Text clustering and NER extraction

Судалгааны өгүүлэл бүрийн abstract-ийн embedding авч clustering хийх бөгөөд нэр үгийг олно. Ерөнхийдөө бол бэлэн модел ашиглан text-г вектор огторгуйд проекц хийнэ.

4. Co-authorship сүлжээ байгуулах

Гараас оноосон түлхүүр үгийн хайлтын илэрцээс олдсон судлаачдын сүлжээг байгуулах. (just for fun)

```
In [ ]: import os
import re
import pickle
import psutil
import pickle
from tqdm import tqdm
from collections import defaultdict
import threading
```

```

from concurrent.futures import ThreadPoolExecutor, as_completed

import numpy as np
import pandas as pd

from transformers import AutoTokenizer, AutoModel, AutoModelForTokenClassification
from transformers import pipeline
from sentence_transformers import SentenceTransformer

from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.feature_extraction.text import TfidfVectorizer

import igraph as ig
import networkx as nx
import chart_studio.plotly
from chart_studio.plotly import plot, iplot

import requests
from bs4 import BeautifulSoup

from jellyfish import jaro_winkler_similarity

import seaborn as sns
import plotly.graph_objs as go
import matplotlib.pyplot as plt

import plotly.io as pio
pio.renderers.default = 'notebook'

import warnings
warnings.filterwarnings('ignore')

pool = ThreadPoolExecutor()
default_workers_threads = pool._max_workers

print(f"CPU count: {os.cpu_count()}")
print(f"Memory GB: {psutil.virtual_memory().total >> 30}")
print(f"Default thread workers: {default_workers_threads}")

```

CPU count: 16

Memory GB: 31

Default thread workers: 20

1. Arxiv

```

In [ ]: num_workers = 12

# Arxiv нэг хуудсанд гарах өгүүлэлийг тоог 200 гэж заасан тул хуудас хооронд 200-р шилжинэ (increment)
start_page = 0
end_page = 4000
increment = 200

# Advanced search дээр хайлтын түлхүүр үгээ оруулна. Энэ тохиолдолд 'audio recognition'
search_term = 'audio+recognition'
base_link = f"https://arxiv.org/search/advanced?advanced=&terms-0-operator=AND&terms-0-term={search_term}&terms-0-field=all&classification-physics_archives=all&classification-include_cross_li

papers = []

```

```

# Хуудас бүрийн мэдээллийг авахад бид олон удаа request үүсгэж байгаа. Иймд нэг удаа үүсгэсэн connection-г олон дахин ашиглах үүднээс
# session тодорхойлно
def scrape_papers(start_num):
    session = requests.Session()

    linkie = f"{base_link}&start={start_num}"

    html_text = session.get(linkie).text
    soup = BeautifulSoup(html_text, 'html.parser')

    papers_sub = soup.find_all('li', class_ = 'arxiv-result')

    papers.extend(papers_sub)

    session.close()

```

```

In [ ]: with ThreadPoolExecutor(max_workers = num_workers) as executor, tqdm(total = (end_page - start_page),
                                     desc = f'Scraping papers from arxiv, [SEARCH TERM]: {search_term}') as pbar:

    futures = [executor.submit(scrape_papers, start_num) for start_num in range(start_page, end_page, increment)]
    for future in as_completed(futures):
        pbar.update(increment)

print(f"Number of papers scraped: {len(papers)}")

```

```

Scraping papers from arxiv, [SEARCH TERM]: audio+recognition: 100%|██████████| 4000/4000 [00:28<00:00, 141.87it/s]
Number of papers scraped: 4000

```

```

In [ ]: def punct(full_text):
    temp = [sent.strip() for sent in re.findall("s+[^.!?]*[.!?]", full_text)]
    temp = re.sub('\.\.', '.', '. '.join(temp))
    temp = re.sub('s+[a-zA-Z]\.', '', temp)

    return temp

arxiv_code = []
titles = []
authors = []
abstracts = []
journal = []
submitted_dates = []
originally_announced_dates = []

patterns = {
    'submitted to': r'submitted to (.+)',
    'Accepted to': r'Accepted to (.+)',
    'Accepted in': r'Accepted in (.+)',
    'accepted by': r'accepted by (.+)',
    'Journal ref': r'Journal ref: (.+)'
}

```

```

In [ ]: for paper in tqdm(papers, desc = 'Scraping fields'):
    # Arxiv code
    paper_arxiv_code = paper.find('p', class_ = 'list-title is-inline-block').text.split('\n')[0]

    # Paper Title
    paper_title = paper.find('p', class_ = 'title is-5 mathjax').text.strip()

    # Authors

```

```

paper_authors = paper.find('p', class_ = 'authors')
paper_authors = ','.join([name.text.strip() for name in paper_authors.find_all('a')])

# Abstract
paper_abstract = paper.find('p', class_ = 'abstract mathjax')
paper_abstract = punct(paper_abstract.find('span', class_ = 'abstract-full has-text-grey-dark mathjax').text)

# Comment, which includes submitted Journals etc.
# Аливаа өгүүлэгийн хувьд хэвлэгдсэн сэтгүүлийн мэдээлэл нь цөөн тооны pattern-ийн дагуу бичигдсэн байсан.
# Иймд энэ хэсэгт pattern бүрийг шалгана.
paper_comment = paper.find('p', class_='comments is-size-7')
paper_journal = np.nan
if paper_comment:
    paper_comment_text = re.sub(r'\s+', ' ', paper_comment.text.strip())
    for keyword, pattern in patterns.items():
        if keyword in paper_comment_text:
            match = re.search(pattern, paper_comment_text)
            if match:
                paper_journal = match.group(1).strip()
                break

# Submission date
# Сүүлд нь бүх хугацааг харсан. Тэгэхэд хугацаа бүр ижил форматтай байсан.
# Иймд ганц удаа split ашиглаж илгээсэн огноог гаргах боломжтой байсныг ойлгосон. Гэхдээ анх ялгаатай pattern-тай огноо байх вий хэмээн болгоомжилж
# бичсэн аргаа үлдээсэн.
submit_date = paper.find('p', class_ = 'is-size-7').text.split(';')[0]
submit_date = re.findall('([0-9]{1,2}\s(?:Jan(?:uary)?|Feb(?:ruary)?|Mar(?:ch)?|Apr(?:il)?|May|Jun(?:e)?|Jul(?:y)?|Aug(?:ust)?|Sep(?:tember)?|Oct(?:ober)?|(Nov|Dec)(?:ember)?))\s, [0-9]{4})')
submit_date = [sent for sent in submit_date[0] if len(sent)][0]

originally_announced_date = paper.find('p', class_ = 'is-size-7').text.strip().split('originally announced ')[-1][:-1]

arxiv_code.append(paper_arxiv_code)
titles.append(paper_title)
authors.append(paper_authors)
abstracts.append(paper_abstract)
journal.append(paper_journal)
submitted_dates.append(submit_date)
originally_announced_dates.append(originally_announced_date)

```

Scraping fields: 100%|██████████| 4000/4000 [00:05<00:00, 782.09it/s]

```

In [ ]: df = pd.DataFrame({'title': titles,
                           'authors': authors,
                           'abstract': abstracts,
                           'Journal': journal,
                           'code': arxiv_code,
                           'submitted_date': submitted_dates,
                           'originally_announced_date': originally_announced_dates})
df['submitted_date'] = pd.to_datetime(df['submitted_date'])
df['originally_announced_date'] = pd.to_datetime(df['originally_announced_date'], errors = 'coerce')

df.head()

```

Out[]:

	title	authors	abstract	Journal	code	submitted_date	originally_announced_date
0	UAVM: Towards Unifying Audio and Visual Models	Yuan Gong,Alexander H. Liu,Andrew Rouditchenko...	Conventional audio-visual models have independ...	NaN	arXiv:2208.00061	2023-02-15	2022-08-01
1	Multiple-hypothesis RNN-T Loss for Unsupervise...	Cong-Thanh Do,Mohan Li,Rama Doddipatla	This paper proposes a new approach to perform ...	Interspeech 2022	arXiv:2207.14736	2022-07-29	2022-07-01
2	Towards Unconstrained Audio Splicing Detection...	Denise Moussa,Germans Hirsch,Christian Riess	Freely available and easy-to-use audio editing...	NaN	arXiv:2207.14682	2023-03-31	2022-07-01
3	Pronunciation-aware unique character encoding ...	Peng Shen,Xugang Lu,Hisashi Kawai	For Mandarin end-to-end (E2E) automatic speech...	NaN	arXiv:2207.14578	2022-07-29	2022-07-01
4	Learning Phone Recognition from Unpaired Audio...	Da-rong Liu,Po-chun Hsu,Yi-chen Chen,Sung-feng...	ASR has been shown to achieve great performanc...	NaN	arXiv:2207.14568	2022-07-29	2022-07-01

In []:

```
# Өгүүлэл бүрийн хувьд хуулсан arxiv code ашиглан өгүүллийн сэдвүүдийг хуулах
paper_all_subjects = []

def get_subjects(paper_arxiv_code, arxiv_base_link = "https://arxiv.org/abs/"):
    session = requests.Session()

    link = arxiv_base_link + paper_arxiv_code
    response = requests.get(link)

    if response.status_code == 200:
        soup = BeautifulSoup(response.text, 'html.parser')
        subject_tag = soup.find('td', class_='tablecell subjects')
        if subject_tag:
            paper_subjects = subject_tag.text.strip()
        else:
            paper_subjects = np.nan
    else:
        paper_subjects = np.nan

    session.close()

    paper_all_subjects.append(paper_subjects)

"""
with ThreadPoolExecutor(max_workers = num_workers) as executor, tqdm(total = len(arxiv_code),
                                desc = f'Scraping subjects from each paper') as pbar:
    futures = [executor.submit(get_subjects, paper_arxiv_code) for paper_arxiv_code in arxiv_code]
    for future in as_completed(futures):
        pbar.update()
"""
df = pd.read_parquet("arxiv_audio+recognition_search_term.parquet")
df.head()
```

Out []:

	title	authors	abstract	Journal	code	submitted_date	originally_announced_date	subjects
0	M&M: Multimodal-Multitask Model Integrating Au...	Long Nguyen-Phuoc,Renald Gaboriau,Dimitri Dela...	This paper introduces the M&M model, a novel m...	Proceedings of the 19th International Joint Co...	arXiv:2403.09451	2024-03-14	2024-03-01	Computation and Language (cs.CL); Sound (cs.SD...
1	Unsupervised Modality-Transferable Video Highl...	Tingtian Li,Zixun Sun,Xinyu Xiao	Identifying highlight moments of raw video mat...	None	arXiv:2403.09401	2024-03-14	2024-03-01	Sound (cs.SD); Computation and Language (cs.CL...
2	More than words: Advancements and challenges i...	Anna Kruspe	This paper addresses the challenges and advanc...	None	arXiv:2403.09298	2024-03-14	2024-03-01	Audio and Speech Processing (eess.AS); Computa...
3	SpeechColab Leaderboard: An Open-Source Platfo...	Jiayu Du,Jinpeng Li,Guoguo Chen,Wei-Qiang Zhang	In the wake of the surging tide of deep learni...	None	arXiv:2403.08196	2024-03-12	2024-03-01	Audio and Speech Processing (eess.AS); Computa...
4	Automatic Speech Recognition (ASR) for the Dia...	Taekyung Ahn,Yeonjung Hong,Younggon Im,Do Hyun...	This study presents a model of automatic speec...	None	arXiv:2403.08187	2024-03-12	2024-03-01	Computer Vision and Pattern Recognition (cs.CV...

In []:

```
#df['subjects'] = paper_all_subjects
df.info(memory_usage = 'deep')

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4000 entries, 0 to 3999
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   title                                4000 non-null   object
1   authors                             4000 non-null   object
2   abstract                             4000 non-null   object
3   Journal                             617 non-null    object
4   code                                4000 non-null   object
5   submitted_date                       4000 non-null   datetime64[ns]
6   originally_announced_date           4000 non-null   datetime64[ns]
7   subjects                             4000 non-null   object
dtypes: datetime64[ns](2), object(6)
memory usage: 6.8 MB

#output_filename = f"arxiv_{search_term}_{search_term}"
#df.to_parquet(f"{output_filename}.parquet")
#df.to_csv(f"{output_filename}.tsv", sep = "\t")
```

2. Google Scholar

In []:

```
# Өгүүлэл бүрийн хувьд хамт ажилласан судлаачдын хүснэгтийг байгуулах.
author_pairs = []

for authors in df['authors']:
    author_list = authors.split(',')
    author_pairs.extend([(author.strip(), other_author.strip()) for i, author in enumerate(author_list) for other_author in author_list[i+1:]])

author_pair_df = pd.DataFrame(author_pairs, columns = ['Author1', 'Author2'])

print(f"Authors network shape: {author_pair_df.shape}")
author_pair_df.head()

Authors network shape: (59140, 2)
```

Out[]:

	Author1	Author2
0	Long Nguyen-Phuoc	Renald Gaboriau
1	Long Nguyen-Phuoc	Dimitri Delacroix
2	Long Nguyen-Phuoc	Laurent Navarro
3	Renald Gaboriau	Dimitri Delacroix
4	Renald Gaboriau	Laurent Navarro

Судлаачдын нэрийг зөв олох

Энэ хэсэгт судлаачдын мэдээллийг үнэн зөв хуулахын тулд судлаачдын нэрийг үнэн зөв тааруулах буюу олох шаардлагатай байсан. Иймд google scholar дээрээс судлаачдын нэрийг тааруулахад **Jaro-Winkler Зай** тооцсон ба гараас босго тогтоосон.

Энэ зай нь гараас оноосон s_1 , s_2 тэмдэгт мөрийн хувьд хоорондоо таарсан тэмдэгтийн жинлэсэн давтамжийг тооцдог.

$$sim_j = \begin{cases} 0 & \text{if } m = 0 \\ \frac{1}{3} \left[\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right] & \text{otherwise} \end{cases}$$

энд:

- $|s_i|$ тэмдэгт мөрийн урт
- m хоорондоо таарсан тэмдэгтийн тоо
- t нийт удаа хийгдсэн шилжилтийн тоо

Тодорхойлолт: s_1 , s_2 -н 2 тэмдэгтүүд нь хоорондоо ижил байх нөхцөл нь:

- Ижил тэмдэгт
- Хоорондоо $\lfloor \frac{\max(|s_1|, |s_2|)}{2} \rfloor$ тэмдэгтээс холгүй зайтай байх

Хоорондоо таарсан тэмдэгтийн жишээ авъя. FAREMVIEL болон FARMVILLE . **F, A, R** ижил байрлалд байна. Харин **M, V, I, E, L** 3 тэмдэгтийн зайд байна. Иймд нийт 8 тэмдэгт хоорондоо таарч байна.

Шилжилтийн тоо гэдэг нь s_1 , s_2 мөрүүдийн хоорондоо таарч байгаа гэхдээ ялгаатай байрлалд байгаа тэмдэгтүүдийн тоог 2-т хуваасан тоо юм. Дээрх мөрүүдийн хувьд ялгаатай байрлалд байгаа тэмдэгт нь **E, L**.

Иймд FAREMVIEL болон FARMVILLE хоорондох зай (төсөө) нь $\frac{1}{3}(\frac{8}{9} + \frac{8}{9} + \frac{8-1}{8}) = 0.88$

Source: [Jaro–Winkler distance](#)

In []:

```
session = requests.Session()

# Unique нэр бүрийн хувьд google scholar дээрээс уг нэрийг оруулан хайлт хийнэ.
# Хайлтаас илэрсэн нэр бүртэй төсөөг нь тооцно. Гараас оруулсан босгыг хангаж байвал нэг хүн гэж үзнэ.
def return_matching_profile_link(base_link, author, cut_off = 0.6):
    author_search_name = author.replace(' ', '+')
    linkie = f'https://scholar.google.com/citations?hl=en&view_op=search_authors&mauthors={author_search_name}&btnG='

    html_text = session.get(linkie).text
    soup = BeautifulSoup(html_text, 'html.parser')

    profile = soup.find_all('div', class_ = 'gsc_1usr')
```

```

jaro_similarity_scores = []
hit_names = []
links = []

if profile:
    for hit in profile:
        hit = hit.find('h3', class_='gs_ai_name').find('a')
        hit_name = hit.text.strip()
        hit_link = base_link + hit['href']

        # Jaro-Winkler мөсөөг тооцох.
        jaro_similarity_score = jaro_winkler_similarity(author, hit_name)

        if jaro_similarity_score >= cut_off:
            jaro_similarity_scores.append(jaro_similarity_score)
            links.append(hit_link)
            hit_names.append(hit_name)

if len(jaro_similarity_scores):
    # Нэгээс олон нэр босго хангах боломжтой учир тэдгээрээс хамгийн өндөр оноотойг сонгох.
    max_score_ind = np.argmax(jaro_similarity_scores)
    max_score_profile_name = hit_names[max_score_ind]
    profile_link = links[max_score_ind]
else:
    profile_link = None
    max_score_profile_name = np.nan

return profile_link, max_score_profile_name

```

```

In [ ]: # Судлаачийг google scholar-с зөв олсон гэж үзэн холбогдох мэдээллүүдийг хуулах.
def return_author_info(profile_link):
    if profile_link:
        user_html_text = session.get(profile_link).text
        user_soup = BeautifulSoup(user_html_text, 'html.parser')

        institute = user_soup.find('div', class_='gsc_prf_il')
        research_area = user_soup.find('div', {"id": "gsc_prf_int"}, class_='gsc_prf_il')

        if institute:
            institute = institute.text.strip()
        if research_area:
            research_area = [item.text.lower().strip() for item in research_area.find_all('a')]
        elif (not institute) and (not research_area):
            institute = np.nan
            research_area = np.nan
    else:
        institute = np.nan
        research_area = np.nan

    return institute, research_area

```

```

In [ ]: hit_names = []
author_institute = []
author_research_area = []
author_unique_names = pd.concat([author_pair_df['Author1'], author_pair_df['Author2']]).unique().tolist()

base_link = 'https://scholar.google.com'

```

```

"""
def process_author(name):
    profile_link, min_score_profile_name = return_matching_profile_link(base_link, name)
    institute, research_area = return_author_info(profile_link)

    if isinstance(research_area, list):
        research_area = ','.join(research_area)

    hit_names.append(min_score_profile_name)
    author_institute.append(institute)
    author_research_area.append(research_area)

with ThreadPoolExecutor(max_workers = num_workers) as executor:
    future_to_author = {executor.submit(process_author, name): name for name in author_unique_names}

    for future in tqdm(as_completed(future_to_author),
                        total = len(future_to_author), desc = "Processing Authors"):
        author_name = future_to_author[future]
        try:
            future.result()
        except Exception as e:
            print(f"Error processing author {author_name}: {e}")
"""

for name in tqdm(author_unique_names, desc = "Processing Authors"):
    profile_link, min_score_profile_name = return_matching_profile_link(name)
    institue, research_area = return_author_info(profile_link)

    if isinstance(research_area, list):
        research_area = ','.join(research_area)

    hit_names.append(min_score_profile_name)
    author_institute.append(institutue)
    author_research_area.append(research_area)

```

Нийт нэрийн тооноос хамааран программ 2-5 цаг ажиллах тохиолдлууд гарч байсан. Иймд судлаачийн мэдээллийг хуулах ажлыг салаалах буюу параллелиар хийхээр шийдсэн.

Edit: Параллелиар хуулалт хийх үед google scholar-руу олон тооны хандалт илгээгдэж байгаа. Үүнийг google bot хэмээн үзэж миний холболтыг хязгаарлаж байгаа тул энэ хэсэгт хуулалтыг нэг нэгээр нь хийхээр босон. Гэсэн хэдий ч миний холболтыг bot хэмээн **flag** хийсэн нь дуусаагүй тул энэ хэсэг боломжгүй болов.

Гэвч яг энэ кодыг ашиглан би 2 жилийн өмнө судлаач бүрийн мэдээллийг Google Scholar-с авч байсан. Энэхүү датаг би тухайн үед Kaggle дээрээ public тохиргоотой оруулсан.

<https://www.kaggle.com/datasets/temuujinerdene/nlpresearchers>

```

In [ ]: # Хуулсан мэдээллүүдээ pickle формат болон хүснэгтээс хадгалах
with open('author_institute.pickle', 'wb') as handle:
    pickle.dump(author_institute, handle, protocol = pickle.HIGHEST_PROTOCOL)
    print("saved to author_institute.pickle")

```

```

In [ ]: with open('author_research_area.pickle', 'wb') as handle:
    pickle.dump(author_research_area, handle, protocol = pickle.HIGHEST_PROTOCOL)
    print("saved to author_research_area.pickle")

```

```
In [ ]: authors_info = pd.DataFrame({'author': author_unique_names,
                                   'hit_name': hit_names,
                                   'institute': author_institute,
                                   'research_area': author_research_area})

print(f"Authors info df shape: {authors_info.shape}")
authors_info.sample(10)
```

```
In [ ]: authors_info.info(memory_usage = 'deep')
```

```
In [ ]: output_filename = f"arxiv_{search_term}_search_term_authors_info"
authors_info.to_parquet(f"{output_filename}.parquet")
```

```
In [ ]: authors_info.to_csv(f"{output_filename}.tsv", sep = '\t')
```

3. Text analysis

Хуулсан судалгааны өгүүлэл бүрийн сэдвүүдийн тооны статистик.

```
In [ ]: def explode_subjects(x):
        pattern = r'([^();]+) \(([w.]+\))'
        matches = re.findall(pattern, x)

        subject_list = []
        key_list = []

        for subject, key in matches:
            subject_list.append(subject.strip())
            key_list.append(key.strip())

        return subject_list, key_list
```

```
In [ ]: all_subjects = []
all_keys = []

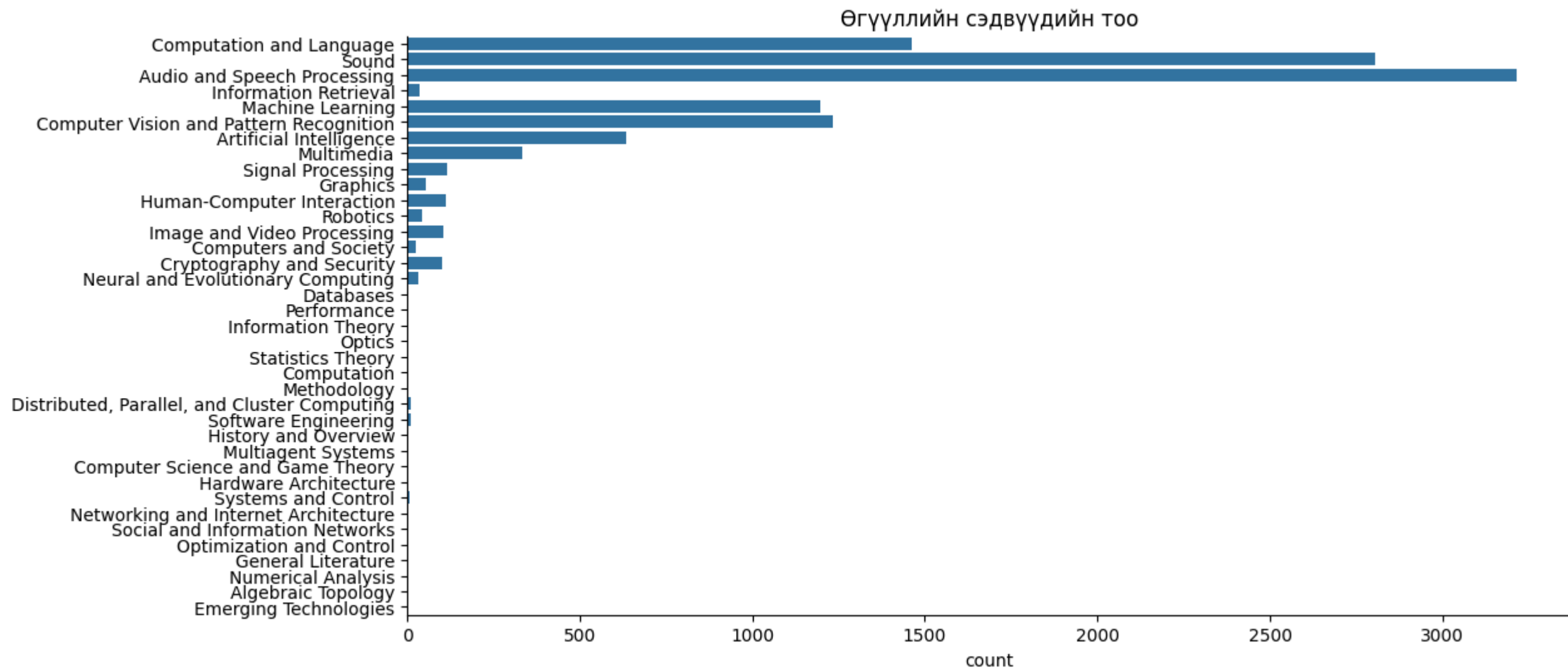
for val in tqdm(df['subjects'].values):
    subject_list, key_list = explode_subjects(val)

    all_subjects.extend(subject_list)
    all_keys.extend(key_list)

fig, ax = plt.subplots(num = 1, clear = True, figsize = (12, 6))
sns.countplot(all_subjects, stat = 'count', ax = ax)
ax.spines[['top', 'right']].set_visible(False)

ax.set_title("Өгүүллийн сэдвүүдийн тоо");
```

100%|██████████| 4000/4000 [00:00<00:00, 152491.03it/s]



3.1 Text Embedding

Бид судалгааны өгүүлэлтэй ажиллаж байгаа тул текст бүрийн вектор буулгалтыг авахын тулд мөн адил судалгааны өгүүлэл ашиглаж сургасан модел ашиглах нь манай бодлогод таарах юм. Иймд энэ хэсэгт би 1.14M өгүүлэл ашиглан сургасан `allenai/scibert_scivocab_uncased` моделийг сонгосон юм.

```
In [ ]: abstracts = df['abstract'].tolist()

# Load the pre-trained model and tokenizer
model_name = "allenai/scibert_scivocab_uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

encoded_abstracts = tokenizer(abstracts, padding = True)

In [ ]: # Векторын хэмжээ нь том тул ерөнхий баримжаа авах үүднээс хэмжээсийг бууруулан дүрсэлсэн нь
pca = PCA(n_components = 2)
pca_data = pca.fit_transform(encoded_abstracts['input_ids'])

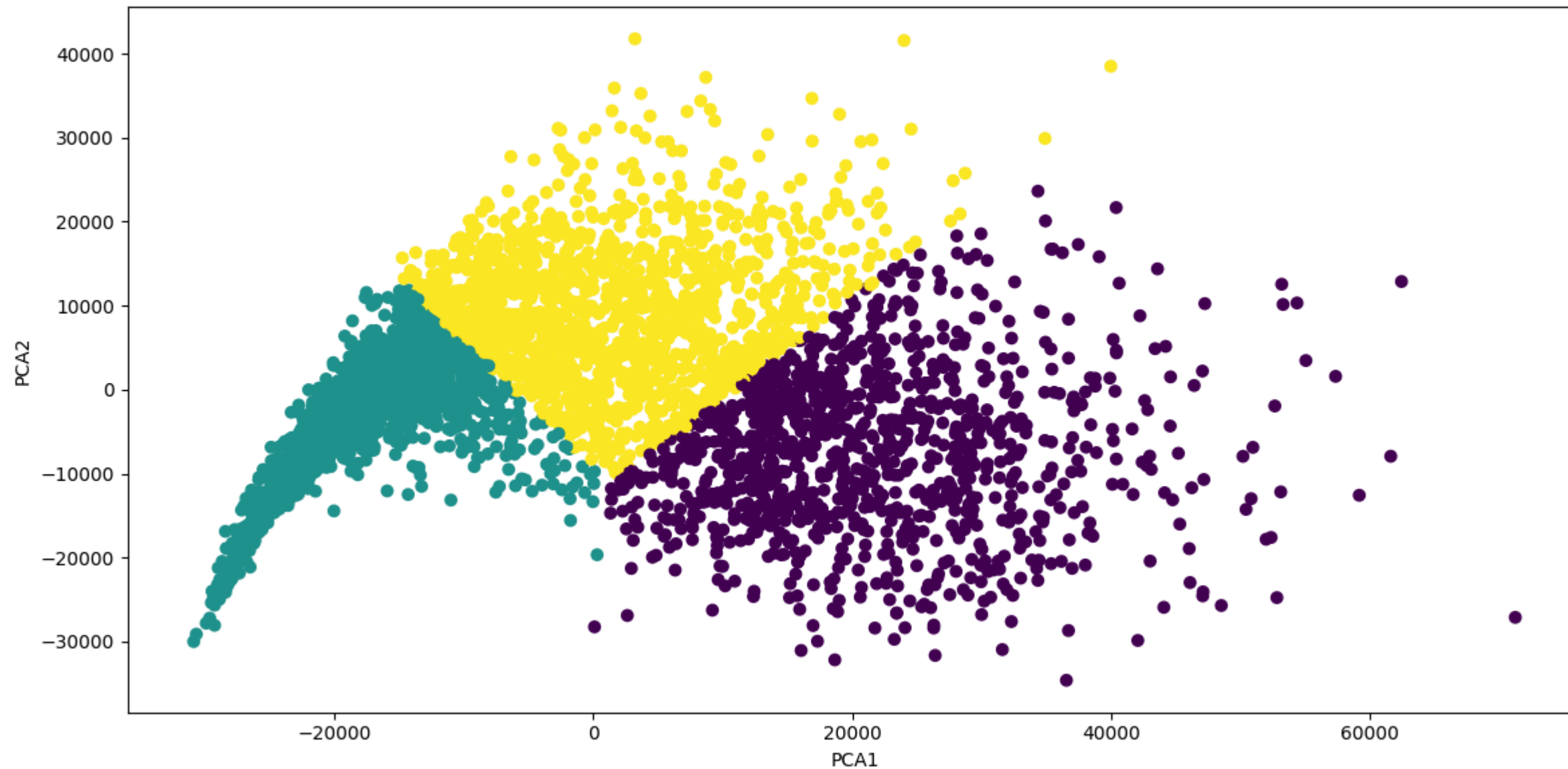
num_clusters = 3
kmeans = KMeans(n_clusters = num_clusters)
kmeans.fit(pca_data)
cluster_labels = kmeans.labels_

fig, ax = plt.subplots(num = 1, clear = True, figsize = (12, 6))
```

```
ax.scatter(pca_data[:, 0], pca_data[:, 1], c = cluster_labels)
ax.set_xlabel("PCA1")
ax.set_ylabel("PCA2")

fig.tight_layout();
```

File "c:\Users\517\anaconda3\envs\num_env\Lib\site-packages\joblib\externals\loky\backend\context.py", line 282, in _count_physical_cores
raise ValueError(f"found {cpu_count_physical} physical cores < 1")



3.2 NER

Энэ хэсэгт мөн адил нэр үг олохын тул судалгааны өгүүлэл ашиглан сургасан модел ашигласан. Энэ [моделийг](#) нь [SciERC](#) өгөгдөл ашиглан сургасан юм билээ.

```
In [ ]: tokenizer = AutoTokenizer.from_pretrained("RJuero/SciNERTopic")
model_trf = AutoModelForTokenClassification.from_pretrained("RJuero/SciNERTopic")

nlp = pipeline("ner", model = model_trf, tokenizer = tokenizer, aggregation_strategy = 'average')
```

```
In [ ]: result_dict = defaultdict(list)

def extract_ner_worker(abstract, lock):
    local_result_dict = defaultdict(list)
    result = nlp(abstract)
    for item in result:
```

```

        entity_group = item['entity_group']
        word = item['word']
        local_result_dict[entity_group].append(word)
        # Өгөгдлийн уралдаан үүсэхээс сэргийлэх
        with lock:
            for key, value in local_result_dict.items():
                result_dict[key].extend(value)

```

```

In [ ]: """
lock = threading.Lock()

with ThreadPoolExecutor(max_workers = 12) as executor, tqdm(total = len(abstracts)) as pbar:
    futures = [executor.submit(extract_ner_worker, abstract, lock) for abstract in abstracts]
    for future in as_completed(futures):
        pbar.update()

file_path = "paper_abstracts_ner_dict.pkl"

with open(file_path, "wb") as file:
    pickle.dump(result_dict, file)
"""

file_path = "paper_abstracts_ner_dict.pkl"

with open(file_path, "rb") as file:
    result_dict = pickle.load(file)

```

```

In [ ]: print(result_dict.keys())

dict_keys(['OtherScientificTerm', 'Task', 'Generic', 'Material', 'Method', 'Metric'])

```

```

In [ ]: fig, axs = plt.subplots(2, 3, num = 1, clear = True, figsize = (20, 15))

for i, category in enumerate(result_dict.keys()):
    category_counts = pd.Series(result_dict[category]).value_counts()[:5]

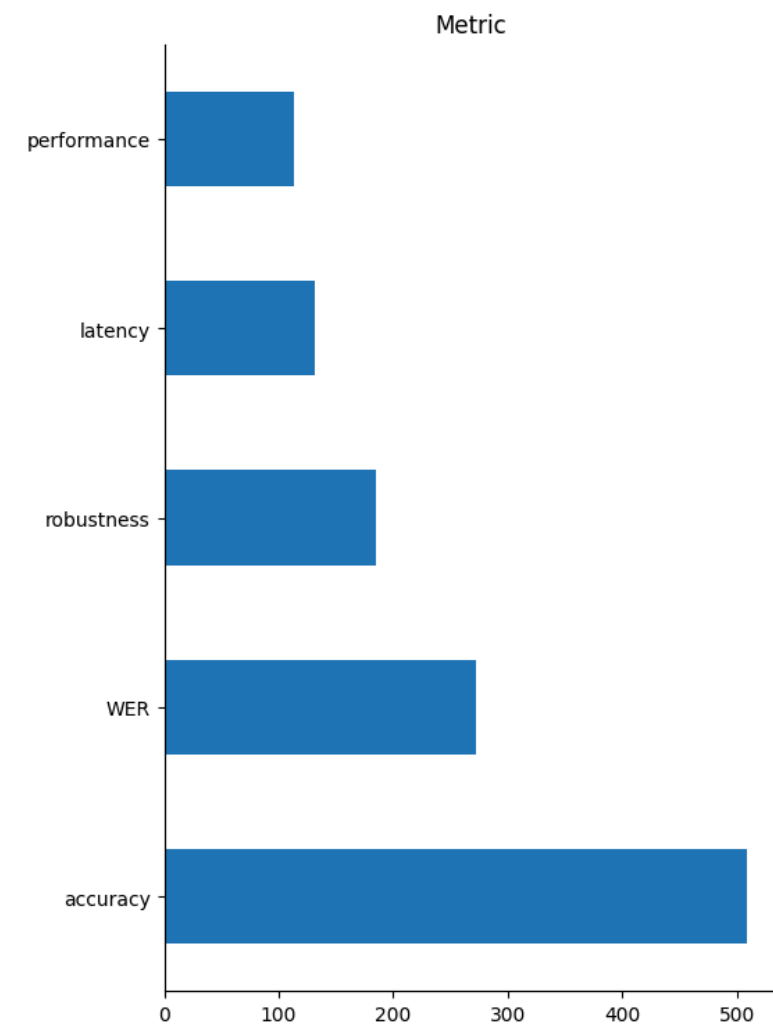
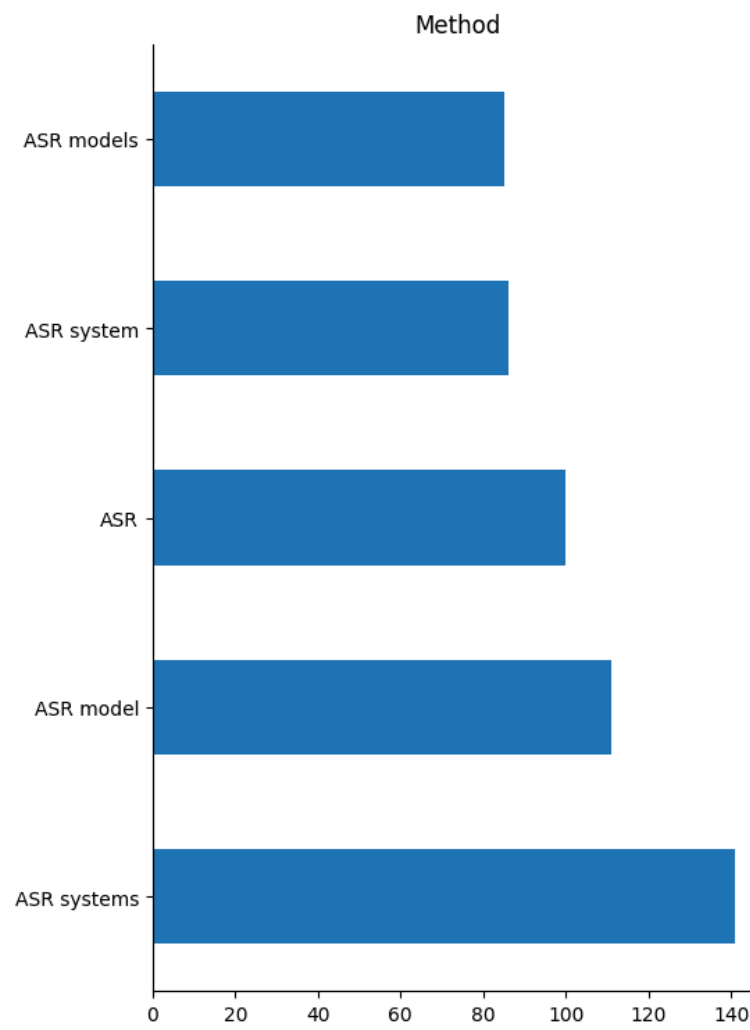
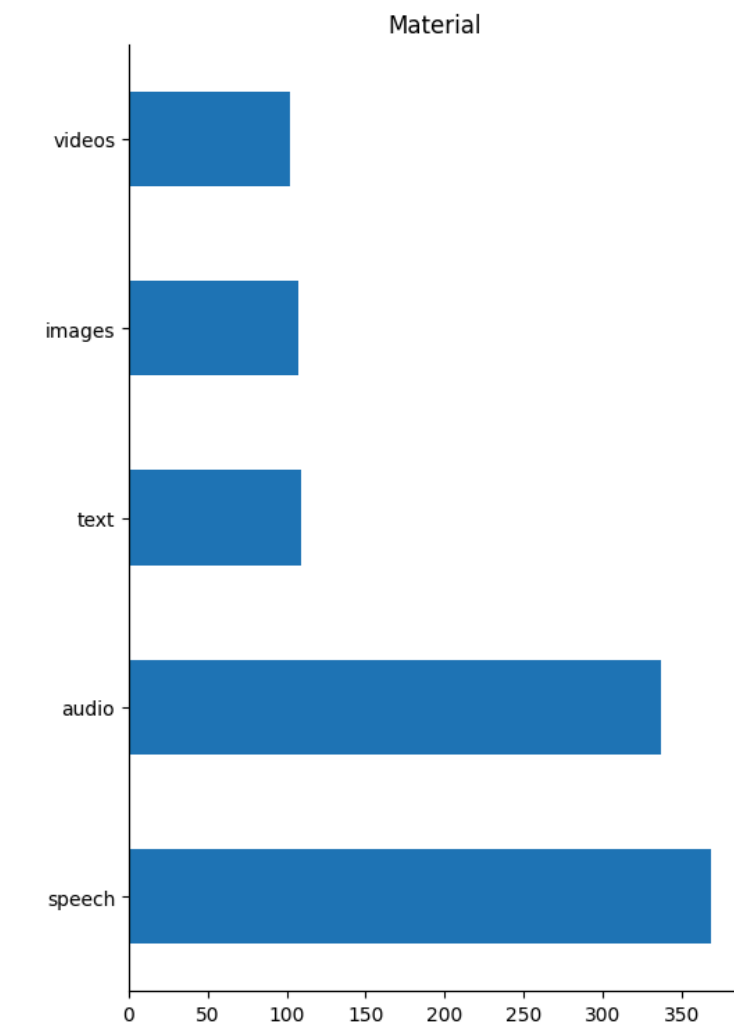
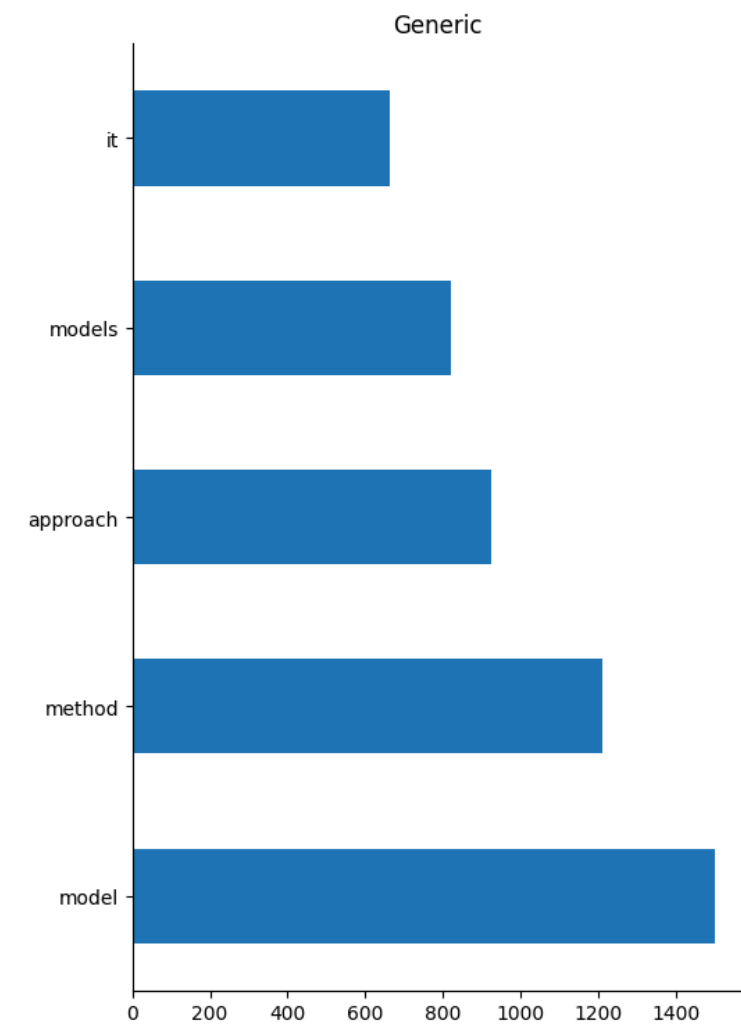
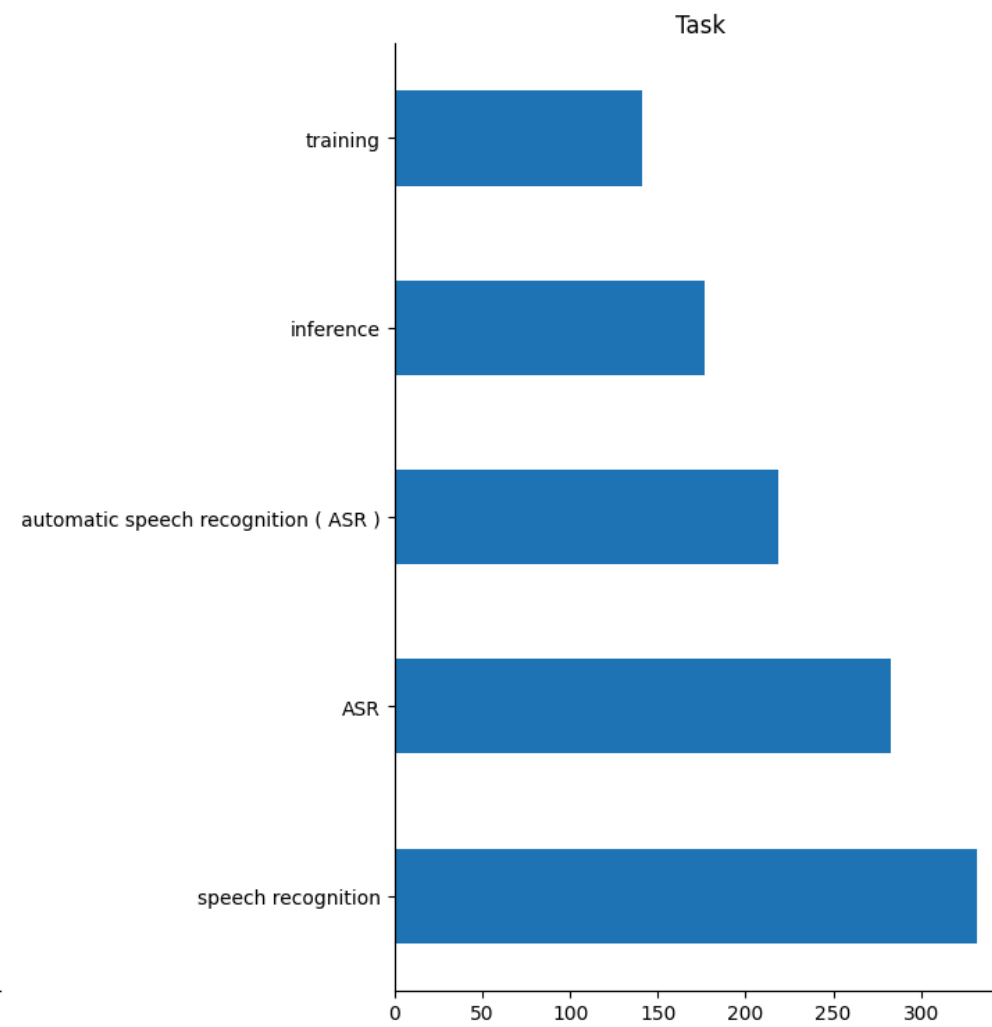
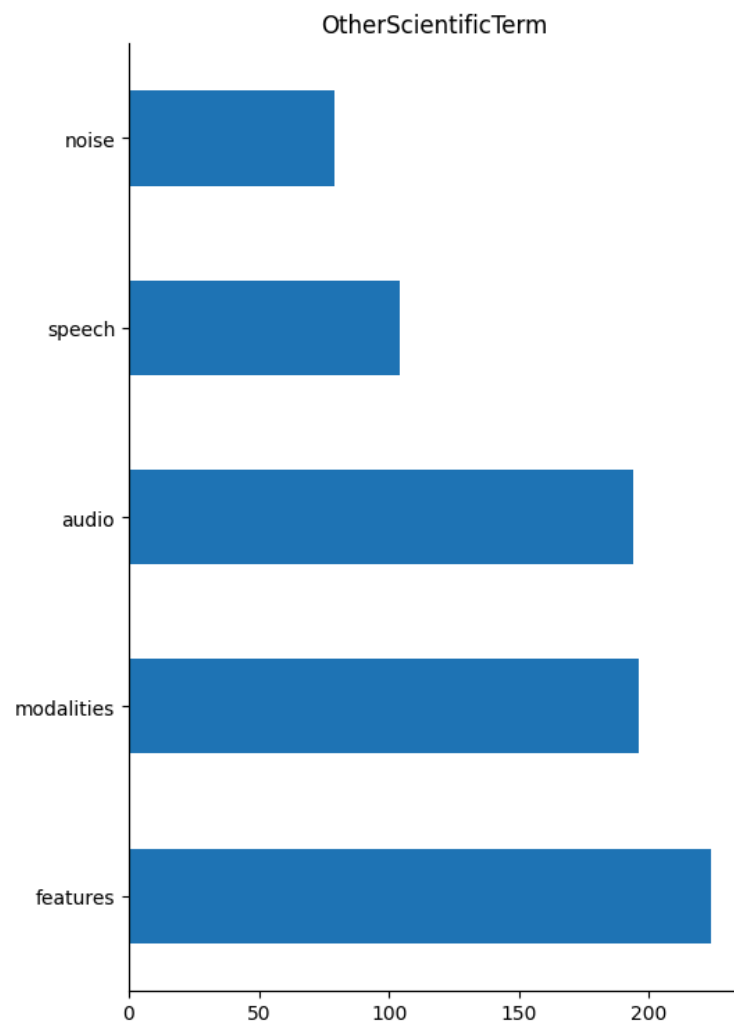
    row = i // 3
    col = i % 3
    ax = axs[row, col]

    category_counts.plot(kind = 'barh', ax = ax)

    ax.set_title(category)
    ax.spines[['top', 'right']].set_visible(False)

fig.tight_layout();

```



4. Co-authorship network

```
In [ ]: sample_num = 5000
author_subset = author_pair_df.head(sample_num)

rn = nx.from_pandas_edgelist(author_subset, source = "Author1", target = "Author2")

print(rn)
```

Graph with 1515 nodes and 4791 edges

```
In [ ]: # Source: https://plotly.com/python/network-graphs/
N = rn.number_of_nodes()
L = rn.number_of_edges()
Edges_name = [e for e in rn.edges()]
Edges = nx.convert_node_labels_to_integers(rn)
Edges = [e for e in Edges.edges()]

# Create an igraph graph
G = ig.Graph(Edges, directed=False)

# Use a 2D layout
layt = G.layout('fruchterman_reingold', dim = 2)

# Extract X, Y coordinates
Xn = [layt[k][0] for k in range(N)]
Yn = [layt[k][1] for k in range(N)]

Xe = []
Ye = []

for e in Edges:
    Xe += [layt[e[0]][0], layt[e[1]][0], None]
    Ye += [layt[e[0]][1], layt[e[1]][1], None]

labels = [e[0] for e in Edges_name]
group = np.repeat(1, 2000).tolist() + np.repeat(2, 2000).tolist() + np.repeat(3, 3000).tolist() + np.repeat(4, 1000).tolist() + np.repeat(5, 2000).tolist()

trace1 = go.Scatter(x = Xe, y = Ye, mode = 'lines',
                    line = dict(color = 'rgb(125,125,125)', width = 1), hoverinfo = 'none')

trace2 = go.Scatter(x = Xn, y = Yn, mode = 'markers', name = 'Researcher',
                    marker = dict(symbol = 'circle', size = 4, color = group, colorscale = 'Viridis',
                                   line = dict(color = 'rgb(50,50,50)', width = 0.5)),
                    text = labels, hoverinfo = 'text')
```

```
In [ ]: layout = go.Layout(
    title = f"Arxiv-c {search_term} түлхүүр үгийн дагуу хайлтаас илэрсэн өгүүллүүдийн хувьд тооцсон судлаачдын танилын сүлжээ. {sample_num} samples",
    autosize = True,
    width = 1200,
    height = 1000,
    showlegend = False,
    scene = dict(),
    margin = dict(t = 100),
    hovermode = 'closest',
    annotations = [
        dict(
```

```
        showarrow = False,
        text = "Гүйцэтгэсэн Э.Тэмүүжин (20B1NUM1970) Хэрэглээний математик 4-р түвшин",
        xref = 'paper',
        yref = 'paper',
        x = 0,
        y = 0,
        xanchor = 'left',
        yanchor = 'bottom',
        font = dict(size = 14)
    )
])

data = [trace1, trace2]
fig = go.Figure(data = data, layout = layout)
fig.show()
```

Arxiv-c audio+recognition түлхүүр үгийн дагуу хайлтаас илэрсэн өгүүллүүдийн хувьд тооцсон судлаачдын танилын сүлжээ. 5000

